AutoPipeAD: Selección Automática de Pipelines de Preprocesamiento para Detección de Anomalías en Imágenes

Ricardo Antonio Borbón Mena
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
ricardobm@estudiantec.cr

Luis Alberto Chavarría Zamora *(Supervisor)*
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
lachavarria@itcr.ac.cr

***Resumen*—La detección de anomalías en imágenes es un problema relevante en inspección industrial, control de calidad y análisis médico. El desempeño de los detectores de anomalías depende en gran medida del preprocesamiento aplicado a las imágenes de entrada, y elegir manualmente esta cadena de transformaciones toma tiempo y requiere conocimiento especializado en visión computacional. En este trabajo se presenta AutoPipeAD, una librería Python de código abierto que automatiza la búsqueda del pipeline de preprocesamiento óptimo para un modelo base de detección de anomalías provisto por el usuario. La librería recibe conjuntos de imágenes organizados en subdirectorios de muestras normales y anómalas, y evalúa sistemáticamente combinaciones de siete transformaciones predefinidas mediante búsqueda exhaustiva por cuadrícula u optimización bayesiana con Optuna (muestreo TPE y poda por mediana). La evaluación paralela se implementó con joblib. Entre las métricas configurables se incluyen AUC-ROC, F1-score, precisión, recall y tasa de falsos positivos; el pipeline ganador se guarda en JSON o YAML para garantizar reproducibilidad. Los experimentos con tres datasets sintéticos comprobaron el correcto funcionamiento de la librería y su compatibilidad con modelos base de scikit-learn y PyTorch.**

***Index Terms*—detección de anomalías, preprocesamiento de imágenes, selección automática de pipelines, búsqueda en cuadrícula, optimización bayesiana, Optuna, procesamiento paralelo, IsolationForest, AUC-ROC, MVTec AD**

I. Introducción

La detección de anomalías en imágenes consiste en identificar automáticamente instancias que se alejan de un patrón esperado, llamado *normal* [1]. Sus aplicaciones van desde la inspección de defectos en superficies industriales hasta el análisis de imágenes médicas y la vigilancia visual. A diferencia de la clasificación supervisada, los detectores de anomalías suelen entrenarse asumiendo que las muestras anómalas son escasas o desconocidas: el modelo aprende la distribución de los casos normales y asigna una puntuación de anomalía a cada instancia nueva [1].

El desempeño de estos detectores depende en buena medida del preprocesamiento aplicado a las imágenes de entrada. Operaciones como el redimensionamiento, el suavizado gaussiano, la normalización estadística o la ecualización adaptativa del histograma cambian la distribución de las características de la imagen, y pueden mejorar o empeorar notablemente la capacidad del modelo para distinguir lo normal de lo anómalo [10]. Elegir esta cadena de transformaciones —el *pipeline de preprocesamiento*— se hace habitualmente a mano, por prueba y error, lo que toma tiempo y exige experiencia en visión computacional.

AutoPipeAD se desarrolló para automatizar este proceso. La librería recibe un dataset de imágenes y un modelo base de detección de anomalías compatible con la interfaz de scikit-learn o PyTorch, y busca automáticamente la combinación de transformaciones que maximiza el desempeño del modelo sobre un conjunto de validación.

El alcance del proyecto se limita a la selección del pipeline de preprocesamiento: no incluye el entrenamiento ni la optimización de la arquitectura del modelo base, ni clasificación multiclase, segmentación semántica, detección de objetos, interfaz gráfica o despliegue como servicio web.

II. Antecedentes

II-A. Detección de Anomalías en Imágenes

Chandola, Banerjee y Kumar [1] hicieron una revisión amplia de los métodos de detección de anomalías, clasificándolos según el nivel de supervisión disponible. En el caso de imágenes, el enfoque más usado es el *aprendizaje de una clase* (*one-class learning*): el modelo se entrena solo con muestras normales y asigna una puntuación de anomalía durante la inferencia.

El detector de vector de soporte de una clase (One-Class SVM, OC-SVM) de Schölkopf et al. [6] estima una hipersuperficie que envuelve la distribución de las muestras normales en el espacio de características. El bosque de aislamiento (Isolation Forest) de Liu, Ting y Zhou [5] sigue una idea distinta: aísla instancias construyendo árboles de decisión al azar, bajo el supuesto de que las anomalías quedan aisladas con menos particiones que las muestras normales. Ambos modelos exponen la interfaz fit/predict de scikit-learn [12], lo que los hace candidatos naturales para usarse con AutoPipeAD.

Métodos más recientes, como PaDiM [4] y PatchCore [3], aprovechan representaciones profundas de redes neuronales preentrenadas para detectar anomalías en imágenes industriales con alta precisión, aunque su integración es más compleja.

El dataset MVTec AD [2] es el conjunto de referencia más usado para evaluar métodos de detección de anomalías en imágenes industriales. Contiene 15 categorías de objetos y texturas con defectos reales, y se usó como referencia de compatibilidad en este trabajo.

II-B. Preprocesamiento de Imágenes

Gonzalez y Woods [10] describieron las transformaciones fundamentales de imágenes digitales. Para la detección de anomalías, las más relevantes son:

- **Redimensionamiento**: uniformiza las dimensiones de las imágenes para garantizar vectores de características de longitud fija.
- **Suavizado gaussiano**: reduce el ruido de alta frecuencia convolucionando la imagen con un núcleo gaussiano de desviación estándar σ.
- **Filtro de mediana**: elimina el ruido impulsivo (sal y pimienta) sin borrar los bordes de la imagen.
- **Normalización estadística**: centra y escala los valores de intensidad según la media μ y la desviación estándar σ del canal, produciendo la imagen transformada $(I-\mu)/\sigma$.
- **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*): mejora el contraste local dividiendo la imagen en bloques y aplicando ecualización de histograma con un límite de contraste (*clip limit*) que evita amplificar el ruido.
- **Corrección gamma**: ajusta la luminosidad global mediante la función de potencia $I_{\text{out}} = I_{\text{in}}^{\gamma}$, donde $\gamma < 1$ aclara la imagen y $\gamma > 1$ la oscurece.
- **Conversión a escala de grises**: reduce la imagen RGB a un solo canal de luminancia, bajando la dimensionalidad del espacio de características.

No existen guías generales para elegir el pipeline de preprocesamiento en detección de anomalías: la combinación óptima depende del tipo de defecto, las condiciones de iluminación y la arquitectura del modelo base.

II-C. Aprendizaje Automático Automatizado y Búsqueda de Pipelines

El aprendizaje automático automatizado (AutoML) estudia cómo seleccionar y configurar de forma automática algoritmos y pipelines de aprendizaje automático [9]. La búsqueda de pipelines de preprocesamiento es un subproblema de AutoML, donde el espacio de búsqueda queda definido por las transformaciones disponibles y sus hiperparámetros, discretos o continuos.

La **búsqueda exhaustiva por cuadrícula** (*grid search*) evalúa el producto cartesiano de todas las opciones discretas. Garantiza explorar el espacio completo, pero su complejidad crece como $O(\prod_{i=1}^{n} |S_i|)$, donde $|S_i|$ es el número de opciones del paso i del pipeline [11].

La **optimización bayesiana** construye un modelo probabilístico de la función objetivo y lo usa para decidir los próximos puntos a evaluar, convergiendo con menos ensayos que la búsqueda aleatoria [8]. Optuna [7] implementa el muestreo por Estimación de Parzen en Árbol (Tree-structured Parzen Estimator, TPE) junto con el podador por mediana (MedianPruner), que descarta ensayos cuyo desempeño cae por debajo de la mediana del estudio tras un número mínimo de evaluaciones.

II-D. Evaluación Paralela

Como la evaluación de cada pipeline candidato es independiente de las demás, esta tarea se presta de forma natural al paralelismo. La librería joblib, distribuida junto con scikit-learn [12], ofrece una interfaz de alto nivel para paralelismo basado en procesos mediante el backend loky, compatible con Windows, Linux y macOS sin necesidad de proteger explícitamente el punto de entrada en scripts Python. La eficiencia de la paralelización se mide con el speedup, definido como el cociente $T_{\text{seq}}/T_{\text{par}}$ entre los tiempos de ejecución secuencial y paralela.

II-E. Métricas de Evaluación

La evaluación de detectores de anomalías se basa en métricas de clasificación binaria. Fawcett [13] describió el área bajo la curva ROC (AUC-ROC) como la métrica más robusta para comparar detectores, porque no depende del umbral de decisión ni del desbalance de clases. El AUC-ROC toma valores entre 0 y 1, donde 1.0 indica clasificación perfecta y 0.5 corresponde a predicción aleatoria. El F1-score, la precisión y el recall complementan el AUC-ROC al considerar un umbral de binarización específico.

El dataset CIFAR-10 [14], con 60 000 imágenes de 32×32 píxeles en diez categorías, se usó como base para construir escenarios de anomalía de una clase contra el resto (one-vs-rest), un protocolo estándar para evaluar detectores no supervisados sobre datos de clasificación.

III. Descripción de la Solución Desarrollada

El repositorio de AutoPipeAD donde se encuentra la implementación completa, la documentación y los ejemplos de uso está disponible en GitHub en la dirección https://github.com/ItsRichard7/AutoPipeAD.

III-A. Arquitectura General

AutoPipeAD se diseñó como una librería Python con seis módulos y interfaces públicas bien definidas. La Figura 1 muestra cómo fluyen los datos entre los componentes del sistema.

III-B. Módulo de Carga de Datos (`data_loader`)

La clase ImageDataLoader cargó imágenes desde un directorio raíz que debió contener los subdirectorios `normal/` (etiqueta 0) y `anomaly/` (etiqueta 1). Se soportaron los formatos `.png`, `.jpg` y `.bmp`; los archivos con extensión no soportada se omitieron con una advertencia de registro. El método load() retornó un arreglo NumPy denso de forma (n, H, W, C) si todas las imágenes compartían dimensiones, o un arreglo de objetos en caso contrario.

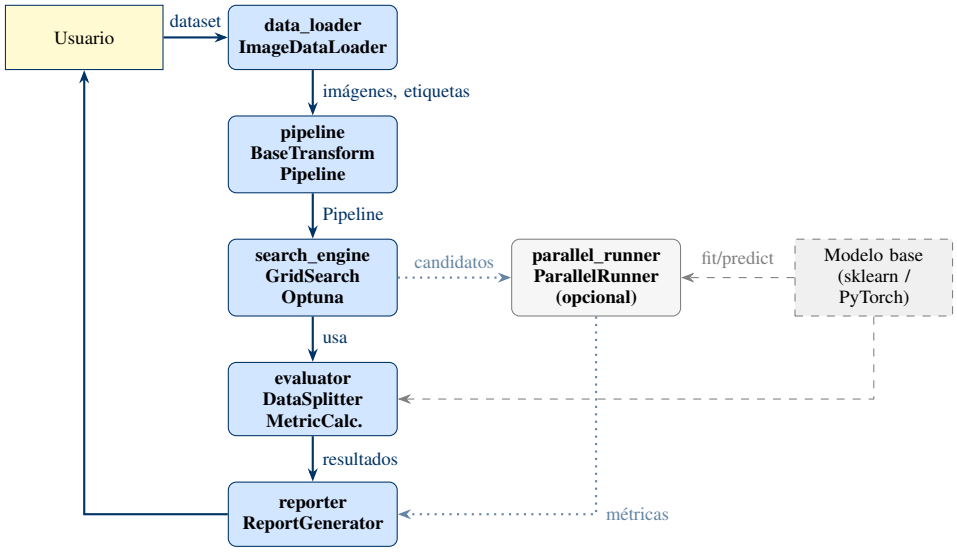


Figura 1. Arquitectura de AutoPipeAD. Las flechas con trazo continuo corresponden al flujo principal de búsqueda automática; las punteadas indican el módulo `parallel_runner`, componente independiente y opcional.

III-C. Módulo de Transformaciones (`pipeline`)

III-C1. Clase base `BaseTransform`: Todas las transformaciones heredan de la clase abstracta `BaseTransform`, que define dos métodos obligatorios:

```
class BaseTransform(ABC):
    @abstractmethod
    def apply(self, image: np.ndarray) -> np.ndarray:
        ...
    @abstractmethod
    def get_params(self) -> dict:
        ...
```

Este contrato permite que el motor de búsqueda trate cualquier transformación de forma uniforme. La Figura 2 muestra la jerarquía de clases del módulo de transformaciones.

III-C2. Transformaciones predefinidas: La librería implementa siete transformaciones: GaussianBlur (suavizado gaussiano con núcleo cuadrado de tamaño impar positivo), MedianFilter (filtro de mediana, kernel $\geq$ 3), Normalize (normalización por media y desviación estándar, produce float32), Resize (redimensionamiento bilineal con OpenCV), CLAHETransform (aplica CLAHE al canal L del espacio LAB en imágenes a color, o directamente en imágenes en escala de grises), GammaCorrection (implementada con una tabla de búsqueda para imágenes enteras) y GrayscaleConversion (conversión de RGB a escala de grises).

III-C3. Clase `Pipeline`: La clase `Pipeline` compone una secuencia ordenada de instancias de `BaseTransform`. Aplica cada transformación en orden, y retorna un diccionario con clave {índice}_{Clase} por paso, lo que permite serializar la configuración ganadora.

III-D. Módulo de Búsqueda (`search_engine`)

El espacio de búsqueda se expresa como un diccionario Python donde cada clave identifica un paso del pipeline y el valor es una lista de transformaciones candidatas. Un ejemplo de uso real con `val_size=0.3`:

```
search_space = {
    "resize":    [Resize(width=32, height=32)],
    "blur":      [GaussianBlur(kernel_size=3),
                  GaussianBlur(kernel_size=5)],
    "normalize": [Normalize(mean=0.5, std=0.5)],
}
engine = GridSearchEngine(
    search_space, val_size=0.3, random_state=42)
best, metrics, results = engine.run(
    images, labels,
    base_model=IsolationForest(
        contamination=0.2, random_state=0),
    metric="auc_roc")
```

Para cada paso del espacio, ambos motores añaden de forma automática y transparente la opción de omitirlo: el usuario no configura nada adicional y la búsqueda también evalúa pipelines sin esa transformación. Un paso omitido simplemente no aparece en el pipeline resultante ni en su configuración serializada, lo que preserva el contrato de tipos de `BaseTransform` y hace trivial la reconstrucción.

III-D1. GridSearchEngine: Genera el producto cartesiano de las listas de candidatos, ampliado con la opción de omisión por paso, y evalúa cada combinación de forma exhaustiva; el espacio efectivo es de $\prod_{i=1}^{n}(|S_i| + 1)$ combinaciones. Para cada pipeline candidato: (1) se aplica la secuencia de transformaciones a cada imagen y se aplana el resultado en un vector de características; (2) se entrena una copia profunda del modelo base sobre el subconjunto de entrenamiento; (3) se calculan las métricas sobre el subconjunto de validación estratificado. Se compara el valor objetivo según la dirección de la métrica: false_positive_rate se minimiza, el resto se maximiza. Los candidatos generados automáticamente que no pueden evaluarse (por ejemplo, omitir el redimensionamiento con imágenes de dimensiones heterogéneas) se registran con métricas NaN y una advertencia, sin abortar la búsqueda; un candidato NaN nunca resulta ganador.

III-D2. OptunaSearchEngine: Implementa optimización bayesiana con TPESampler (semilla fija) y MedianPruner [7].

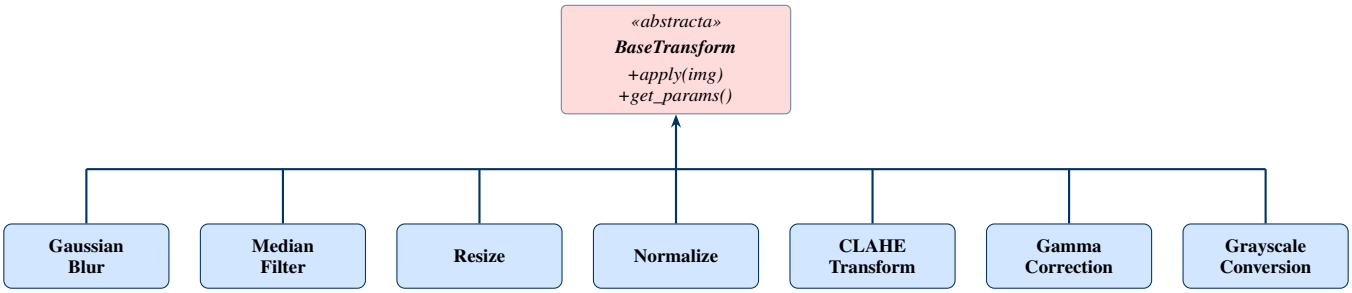


Figura 2. Jerarquía de clases del módulo `pipeline`. Las siete transformaciones concretas heredaron directamente de `BaseTransform` e implementaron `apply()` y `get_params()`. La barra colectora representa herencia múltiple desde un único padre.

Acepta el mismo formato de espacio de búsqueda que GridSearchEngine y expone la misma firma; la omisión de cada paso se modela como un índice categórico adicional en el muestreador. Los valores NaN que producen los ensayos con una sola clase en el conjunto de validación (o los candidatos inevaluables) se reemplazan por $\pm\infty$ para no corromper el estudio de Optuna.

III-E. Módulo de Evaluación (`evaluator`)

III-E1. DataSplitter: Genera particiones train/val estratificadas de scikit-learn [12]. La estratificación garantiza que la proporción de muestras anómalas sea equivalente en ambos subconjuntos, algo importante en datasets desbalanceados. También soporta validación cruzada k-pliegues. La configuración del particionador se puede guardar y recargar en JSON o YAML.

III-E2. MetricCalculator: Calcula AUC-ROC, F1-score, precisión, recall y tasa de falsos positivos. Para obtener las puntuaciones continuas que requiere el AUC-ROC, se prueban en orden los métodos predict_proba, score_samples (negado) y decision_function (negado) del modelo base. Las predicciones de IsolationForest (convención -1=anomalía, $+1$=normal) se normalizan a etiquetas $\{0, 1\}$.

III-F. Módulo de Ejecución Paralela (`parallel_runner`)

La clase `ParallelRunner` evalúa una lista de pipelines de forma concurrente. Cada proceso evalúa un pipeline sobre una copia independiente del modelo base, sin estado compartido mutable. El método benchmark() mide el speedup como $T_{\text{seq}}/T_{\text{par}}$, y verifica que los resultados paralelos y secuenciales sean numéricamente equivalentes.

III-G. Módulo de Reportes (`reporter`)

La clase `ReportGenerator` siempre produce un reporte HTML, con exportación opcional a PDF. Si el binario no está disponible, el sistema registra una advertencia y conserva el HTML (degradación elegante). Las gráficas se generan con matplotlib y se incrustan como URI de datos en base64, lo que produce un documento HTML autocontenido. La configuración del pipeline ganador se guarda en JSON con la estructura:

```
{
  "0_Resize":       {"width": 32, "height": 32},
  "1_GaussianBlur": {"kernel_size": 3},
  "2_Normalize":    {"mean": 0.5, "std": 0.5}
}
```

El método estático rebuild_pipeline(config) reconstruye el pipeline a partir de este archivo, lo que permite repetir exactamente la misma ejecución; los pasos omitidos por la búsqueda no aparecen en la configuración y los índices de los presentes quedan contiguos.

Si el usuario no configura un directorio de salida, los archivos se escriben en una subcarpeta reports/ junto al script que invoca la librería; en sesiones interactivas (Jupyter) se usa el directorio de trabajo, que coincide con el del cuaderno. La carpeta se crea automáticamente, de modo que el caso de uso estándar no requiere configurar rutas.

III-H. Flujo de Evaluación de un Pipeline

La Figura 3 muestra el diagrama de actividad del proceso de evaluación de un pipeline candidato dentro del motor de búsqueda.

III-I. Decisiones de Diseño

Inyección del modelo base: el motor de búsqueda recibe el modelo base como argumento y lo clona antes de cada evaluación. Así se garantiza que todos los candidatos se evalúen sobre un modelo sin entrenar idéntico, preservando la comparabilidad y la reproducibilidad.

Funciones a nivel de módulo para lógica compartida: las funciones `evaluate_pipeline()`, `is_better()` y `validate_metric()` se definieron a nivel de módulo, de modo que `GridSearchEngine`, `OptunaSearchEngine` y `ParallelRunner` pudieran usarlas sin necesidad de herencia.

Elección de joblib sobre multiprocessing: se eligió joblib porque abstrae las diferencias entre plataformas (modelo fork en Linux versus spawn en Windows), es compatible con ambos motores de búsqueda y es la misma dependencia que usa scikit-learn [12].

Elección de Jinja2 + pdfkit: se descartó WeasyPrint porque requiere la pila gráfica GTK (Pango/Cairo), no disponible en Windows sin configuración compleja. En su lugar se adoptó pdfkit con tolerancia a fallos.

Omisión de pasos por ausencia: se eligió representar el paso omitido como ausencia (no aparece en el pipeline ni en su configuración serializada) en lugar de un marcador explícito o una transformación identidad: así se preserva el contrato de tipos de `BaseTransform` y la reconstrucción

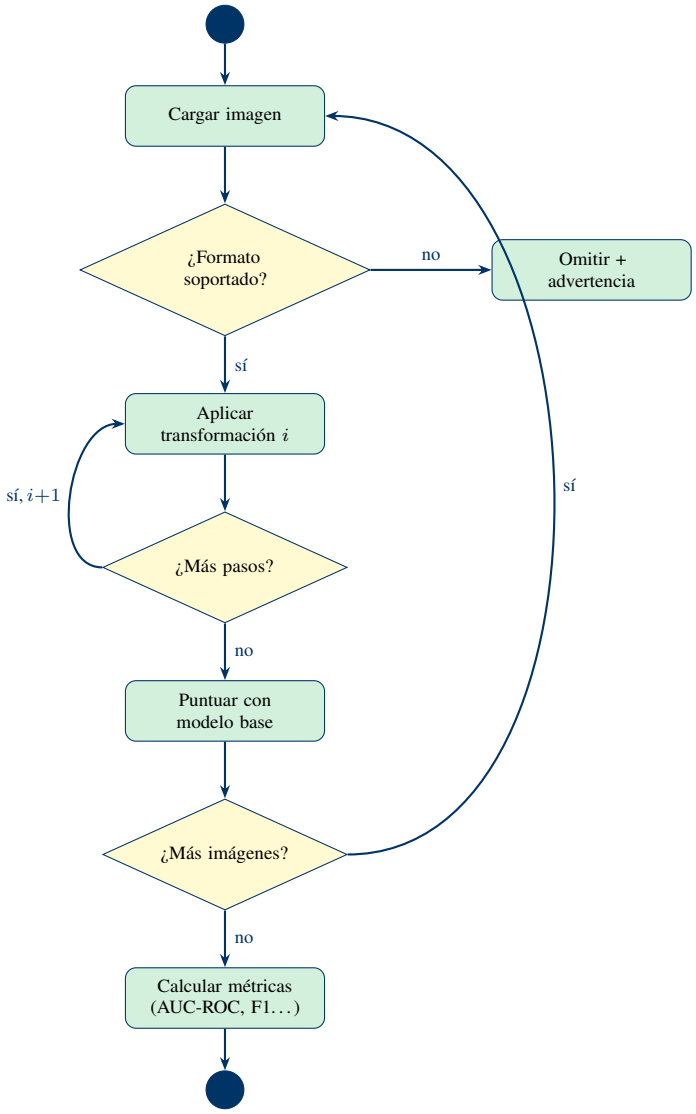


Figura 3. Diagrama de actividad de la evaluación de un pipeline candidato. Este flujo se ejecutó para cada combinación generada por el motor de búsqueda.

desde JSON/YAML funciona sin casos especiales, incluido el pipeline vacío.

III-J. Tecnologías Utilizadas

La Tabla I resume las dependencias principales.

El entorno de desarrollo utilizó Python 3.12 con las herramientas Ruff (verificador de estilo e importaciones) y mypy (verificación estática de tipos estricta).

IV. Resultados

IV-A. Suite de Pruebas

Se desarrolló una suite de pruebas con *pytest*, compuesta por seis archivos de pruebas unitarias e integración —uno por módulo— y un archivo de pruebas de extremo a extremo que verifica el flujo completo, desde la carga del dataset hasta la generación del reporte y la reconstrucción del pipeline ganador:

Cuadro I
Dependencias principales de AutoPipeAD.

Componente	**Librería**	**Versión mínima**
Procesamiento de imágenes	OpenCV	4.9
	scikit-image	0.22
	Pillow	10.0
Cómputo numérico	NumPy	1.26
Modelos ML e interfaz	scikit-learn	1.4
	PyTorch	2.2
Búsqueda bayesiana	Optuna	3.6
Paralelismo	joblib	1.3
Visualización	matplotlib	3.8
Plantillas HTML	Jinja2	3.1
Exportación PDF	pdfkit	1.0
Serialización	PyYAML	6.0

Cuadro II
Configuración del Experimento 1 (dataset sintético).

Parámetro	**Valor**
Imágenes totales	50 (40 normales, 10 anómalas)
Dimensiones	$64 \times 64 \times 3$
`val_size`	0.30
`random_state`	42
Modelo base	IsolationForest (`contam=0.2`, `random_st=0`)
Métrica objetivo	AUC-ROC
Paso *resize*	1 opción: Resize(32×32)
Paso *blur*	2 opciones: GaussianBlur($k = 3$), GaussianBlur($k = 5$)
Paso *normalize*	1 opción: Normalize($\mu = 0.5$, $\sigma = 0.5$)
Total (Grid Search)	12 combinaciones ($2 \cdot 3 \cdot 2$, con omisión por paso)
Ensayos (Optuna)	8

134 pruebas en total con una cobertura del 97 % del código fuente según pytest-cov.

IV-B. Experimento 1: Dataset Sintético con Grid Search y Optuna

El primer notebook ejecutó la búsqueda sobre un dataset sintético de 50 imágenes de $64 \times 64 \times 3$ píxeles, generado de forma programática: las 40 imágenes normales se crearon con valores en el rango $[0, 40]$ (oscuras) y las 10 anómalas con valores en $[200, 256]$ (brillantes), de modo que las anomalías representan un 20 % del conjunto, acorde con el supuesto de rareza de los detectores de anomalías. El modelo base fue `IsolationForest` de scikit-learn [12] con `contamination=0.2` (igual a la proporción real de anomalías) y `random_state=0`. El espacio de búsqueda y los parámetros de evaluación se muestran en la Tabla II.

La búsqueda exhaustiva evaluó las 12 combinaciones y todas —incluidos los subconjuntos generados por la omisión automática y el pipeline vacío— alcanzaron métricas perfectas (Tabla III): la separación de brillo entre clases es tan marcada que cualquier preprocesamiento (o ninguno) preserva la separabilidad, y el umbral calibrado con `contamination=0.2` produce predicciones binarias correctas. El ganador fue el primer candidato en alcanzar el óptimo según el orden de

Cuadro III
RESULTADOS DE GRID SEARCH — EXPERIMENTO 1 (EXTRACTO; LAS 12 COMBINACIONES OBTUVIERON MÉTRICAS IDÉNTICAS).

Pipeline	**AUC-ROC**	**F1**	**Prec.**	**Recall**	**FPR**
#0 Res+Blur+Norm (**winner**)	1.0	1.0	1.0	1.0	0.0
#11 (pipeline vacío)	1.0	1.0	1.0	1.0	0.0

Cuadro IV
CONFIGURACIÓN DEL EXPERIMENTO 2 (CIFAR-10 SINTÉTICO).

Parámetro	**Valor**
Imágenes totales	100 (60 normales, 40 anómalas)
Dimensiones	$32 \times 32 \times 3$
`val_size`	0.30
`random_state`	42
Motor	Optuna (12 ensayos, TPE + MedianPruner)
Paso *resize*	1 opción: Resize(16×16)
Paso *median*	1 opción: MedianFilter($k = 3$)
Paso *blur*	2 opciones: GaussianBlur($k = 3$), GaussianBlur($k = 5$)
Paso *clahe*	1 opción: CLAHE (clip 2.0, rejilla 4×4)
Paso *gamma*	2 opciones: $\gamma = 0.5$, $\gamma = 1.5$
Paso *gray*	1 opción: GrayscaleConversion
Paso *normalize*	1 opción: Normalize($\mu = 0.5$, $\sigma = 0.5$)
Espacio total	288 combinaciones (con omisión por paso)

exploración: Resize(32×32) $\rightarrow$ GaussianBlur($k = 3$) $\rightarrow$ Normalize.

Durante el desarrollo se observó que una versión balanceada 50/50 de este mismo dataset degradaba el AUC-ROC del mejor pipeline a 0.5000 (predicción aleatoria): IsolationForest asume que las anomalías son raras y, con la clase anómala representando la mitad del entrenamiento, el modelo llegó a clasificar todas las muestras de validación como anómalas (FPR=1.0). Por ello el experimento final usa una proporción realista de 20 % de anomalías y el parámetro `contamination=0.2`, que calibra el umbral de decisión binaria a la tasa de anomalías esperada.

La búsqueda bayesiana con Optuna ejecutó 8 ensayos y alcanzó el mismo AUC-ROC de 1.0000 con un pipeline distinto pero igualmente óptimo (GaussianBlur($k = 3$) $\rightarrow$ Normalize, omitiendo el redimensionamiento): cuando varios candidatos comparten el valor óptimo, cada motor retiene el primero que lo alcanza según su propio orden de exploración.

La reproducibilidad se verificó así: la configuración del pipeline ganador se guardó automáticamente, y el método rebuild_pipeline() reconstruyó exactamente el mismo pipeline, confirmado al comparar los parámetros de ambos.

IV-C. Experimento 2: CIFAR-10 Sintético con Optuna

El segundo notebook construyó un escenario de anomalía de una clase contra el resto sobre un dataset CIFAR-10 sintético [14]: las imágenes normales (clase 1) se generaron con tonos verdosos ($\mu = [60, 150, 60]$, $\sigma = 12$) y las anómalas (demás clases) con tonos rojizos ($\mu = [170, 60, 60]$, $\sigma = 40$). La Tabla IV muestra la configuración del experimento.

Cuadro V
MÉTRICAS DEL PIPELINE GANADOR — EXPERIMENTO 2 (CIFAR-10 SINTÉTICO).

Métrica	**Valor**
AUC-ROC	1.0
F1-score	1.0
Precisión	1.0
Recall	1.0
Tasa falsos positivos	0.0

Este experimento usa un espacio de búsqueda completo con las siete transformaciones predefinidas de la librería. De los 12 ensayos de Optuna, 11 alcanzaron AUC-ROC=1.0000 (el restante, 0.9954). El pipeline ganador correspondió al primer ensayo evaluado: MedianFilter($k = 3$) $\rightarrow$ GaussianBlur($k = 3$) $\rightarrow$ CLAHE(2.0) $\rightarrow$ GrayscaleConversion (Tabla V), un pipeline que *omite* tres de los siete pasos disponibles (redimensionamiento, corrección gamma y normalización), lo que ilustra la exploración automática de subconjuntos del espacio de búsqueda.

El AUC-ROC=1.0000 refleja la alta separabilidad del dataset sintético: la diferencia cromática entre la clase normal (verde) y la anómala (rojo) fue suficiente para que IsolationForest separara perfectamente ambas clases con casi cualquier subconjunto de transformaciones. Este resultado confirma la correctitud funcional del sistema sobre un espacio amplio (288 combinaciones exploradas con solo 12 ensayos) y muestra que la omisión automática de pasos permite encontrar pipelines más cortos con el mismo desempeño.

IV-D. Experimento 3: Adaptador para MVTec AD

El tercer notebook mostró la integración de AutoPipeAD con el dataset MVTec AD [2], cuya estructura de directorios (categorías con subdirectorios por tipo de defecto) es distinta del formato que espera `ImageDataLoader`. El notebook incluye la función `build_normal_anomaly_dir()`, que convierte la estructura `test/good/` + `test/`*`defecto`*`/` al formato `normal/` + `anomaly/`. Como no se contó con el dataset real (requiere registro y descarga de $\sim$4.9 GB), el notebook se probó con un dataset sintético de validación (30 imágenes normales con textura gris uniforme y 18 anómalas con parche brillante), lo que confirmó que el adaptador funciona correctamente. El espacio de búsqueda usó `CLAHETransform` con clip_limit $\in \{2.0, 4.0\}$ y `GaussianBlur` con $k \in \{3, 5\}$, junto con `Resize`(64×64) y `Normalize`; con la omisión automática por paso, la búsqueda exhaustiva evaluó 36 combinaciones ($2 \cdot 3 \cdot 3 \cdot 2$). De ellas, 24 alcanzaron AUC-ROC=1.0000 y la peor obtuvo 0.8704; el ganador fue Resize(64×64) $\rightarrow$ CLAHE(2.0) $\rightarrow$ GaussianBlur($k = 3$) $\rightarrow$ Normalize, con las cinco métricas perfectas.

IV-E. Evaluación del Módulo de Paralelismo

El método `ParallelRunner.benchmark()` ejecutó la evaluación del espacio de búsqueda del Experimento 1 (12 pipelines, incluida la omisión por paso) en modo secuencial

Cuadro VI
RESULTADOS DE LA EVALUACIÓN DE PARALELISMO — EXPERIMENTO 1.

Parámetro	Valor medido
Speedup medido	0.25×
Consistencia (seq. vs. par.)	Verdadero
n_jobs	−1 (todos los núcleos)
Pipelines evaluados	12

Cuadro VII
COMPARACIÓN GRID SEARCH VS. OPTUNA — EXPERIMENTO 1.

Estrategia	Evaluaciones	AUC-ROC ganador	Pipeline ganador
Grid Search	12	1.0000	Resize + Blur + Norm
Optuna	8	1.0000	Blur + Norm

(`n_jobs=1`) y en modo paralelo (`n_jobs=-1`), obteniendo los resultados de la Tabla VI.

El *speedup* de 0.25× (la ejecución paralela fue alrededor de 4 veces más lenta que la secuencial) es el comportamiento esperado para un espacio de búsqueda pequeño de evaluación barata: la sobrecarga de inicializar el *pool* de procesos *loky* domina sobre el tiempo de cómputo de cada pipeline, aunque la penalización se reduce al crecer el espacio (con 2 pipelines, en mediciones previas, el *speedup* fue de 0.06×). El beneficio de la paralelización aparece en espacios más amplios o con modelos base costosos, donde el tiempo de evaluación supera esa sobrecarga. El indicador `consistent=True` confirma que los resultados paralelos y secuenciales fueron numéricamente equivalentes, lo que valida la correctitud del mecanismo.

IV-F. Comparación de Estrategias de Búsqueda

La Tabla VII compara las dos estrategias de búsqueda en el Experimento 1, donde el espacio fue lo bastante pequeño para evaluar ambas sobre el mismo conjunto.

Ambas estrategias alcanzaron el mismo AUC-ROC ganador, y Optuna lo hizo con menos evaluaciones (8 frente a las 12 del espacio completo). Las composiciones ganadoras difieren porque en este dataset varios candidatos comparten el valor óptimo y cada motor retiene el primero que lo alcanza según su orden de exploración; ambas son igualmente válidas. La ventaja de la optimización bayesiana se acentúa en espacios con cientos de combinaciones, como el del Experimento 2 (288 combinaciones exploradas con 12 ensayos).

IV-G. Fortalezas de la Solución

1. **Reproducibilidad garantizada**: la semilla fija en el particionador y en el muestreador de Optuna produce resultados idénticos en ejecuciones independientes, verificado con el pipeline reconstruido.
2. **Exploración automática de subconjuntos**: cada paso del espacio incluye la opción transparente de omitirse, sin configuración adicional del usuario; la búsqueda puede así descartar transformaciones que no aportan (el ganador del Experimento 2 omitió tres de los siete pasos disponibles).
3. **Extensibilidad**: la interfaz `BaseTransform` permite agregar transformaciones personalizadas sin modificar ningún otro módulo.
4. **Compatibilidad con la interfaz estándar**: la librería acepta cualquier modelo con métodos fit/predict de scikit-learn, así como modelos PyTorch con la misma interfaz.
5. **API concisa**: el flujo completo de carga, búsqueda y reporte se expresa en menos de 20 líneas de código de usuario, según muestran los notebooks de ejemplo.
6. **Reporte autocontenido y de ubicación predecible**: el documento HTML incluye gráficas e imágenes de anomalías codificadas en base64, sin depender de archivos externos; sin configuración, los archivos se escriben junto al script o cuaderno invocador, con la carpeta creada automáticamente.
7. **Adaptador de estructura de datos**: Se demuestra que la librería puede integrarse con datasets de formato distinto mediante adaptadores externos.

IV-H. Limitaciones de la Solución

1. **Sensibilidad al balance de clases**: IsolationForest asume que las anomalías son raras; en pruebas de desarrollo con un balance 50/50 en el entrenamiento, el AUC-ROC bajó a 0.5000 (predicción aleatoria). El Experimento 1 final mitiga esta limitación del modelo base con una proporción realista (20 % de anomalías) y el parámetro contamination acorde.
2. **Paralelismo no beneficioso para espacios pequeños**: el *speedup* de 0.25× con 12 pipelines de evaluación barata muestra que la paralelización resulta contraproducente cuando la sobrecarga de crear los procesos supera el costo de cada evaluación.
3. **Representación por aplanamiento de píxeles**: la extracción de características aplana la imagen en un vector, lo que descarta la estructura espacial local. Esto limita la compatibilidad con modelos basados en características profundas, como PatchCore o PaDiM.
4. **Validación simple**: la búsqueda usa una sola partición train/val, lo que introduce varianza en la estimación de la métrica para datasets pequeños. La validación cruzada k-pliegues ya está implementada, pero todavía no se integró en el bucle de búsqueda automática.
5. **Exportación a PDF condicional**: generar el PDF requiere el binario externo wkhtmltopdf, que no viene incluido entre las dependencias de la librería.

V. CONCLUSIONES

AutoPipeAD implementa un sistema funcional y reproducible para seleccionar automáticamente pipelines de preprocesamiento en detección de anomalías en imágenes. La arquitectura modular de seis componentes, con interfaces bien definidas, separa las responsabilidades de carga de datos, definición de transformaciones, búsqueda de configuraciones, evaluación de métricas, ejecución paralela y generación de reportes, lo que permite extender la librería sin modificar componentes existentes.

Los experimentos confirmaron la correctitud funcional del sistema. En el Experimento 1, las dos estrategias de búsqueda alcanzaron un AUC-ROC de 1.0000 de forma determinista bajo una semilla fija: Grid Search evaluó las 12 combinaciones del espacio (incluidos los subconjuntos generados por la omisión automática de pasos) y Optuna llegó al mismo óptimo con solo 8 ensayos. Reconstruir el pipeline desde el archivo JSON generado dio resultados idénticos, y las métricas de los tres notebooks resultaron reproducibles entre ejecuciones independientes. En el Experimento 2, la búsqueda bayesiana identificó desde el primer ensayo un pipeline con AUC-ROC de 1.0000 sobre un espacio de 288 combinaciones con las siete transformaciones predefinidas, omitiendo tres pasos que no aportaban al desempeño.

La evaluación paralela mediante benchmark para 12 pipelines dio un de $0.25\times$, lo que muestra que la sobrecarga del backend superó el beneficio computacional en espacios de búsqueda de evaluación barata. La consistencia entre resultados paralelos y secuenciales se verificó en todos los experimentos.

El hallazgo más relevante durante el desarrollo fue la sensibilidad de IsolationForest al balance de clases en el conjunto de entrenamiento: con una proporción 50/50 de muestras normales y anómalas, el detector operó al nivel de una predicción aleatoria (AUC-ROC=0.5). Esta limitación es propia del modelo base, no del sistema de selección de pipelines, y se mitigó en los experimentos finales usando una proporción realista de anomalías (20 %) junto con el parámetro contamination calibrado a esa tasa.

VI. Recomendaciones y Trabajo Futuro

VI-A. Integración de Validación Cruzada en el Bucle de Búsqueda

Incorporar la validación cruzada k-pliegues (ya implementada en `DataSplitter`) dentro del bucle de búsqueda automática reduciría la varianza al estimar métricas en datasets pequeños. La métrica objetivo podría promediarse sobre los k pliegues en lugar de evaluarse sobre una única partición.

VI-B. Extracción de Características Profundas

Agregar un extractor de características basado en redes neuronales convolucionales preentrenadas —como ResNet o los descriptores de parches de PatchCore como etapa adicional del pipeline aumentaría la capacidad discriminativa sin necesitar etiquetas de anomalías durante el entrenamiento, superando la limitación del aplanamiento de píxeles.

VI-C. Soporte de Hiperparámetros Continuos

Los hiperparámetros continuos del preprocesamiento —como el valor σ del suavizado gaussiano o el exponente γ— podrían incorporarse al espacio de búsqueda de Optuna como variables reales, aprovechando la capacidad nativa del muestreador TPE para espacios mixtos.

VI-D. Interfaz de Línea de Comandos

Desarrollar una interfaz de línea de comandos (CLI) que lea la configuración desde el archivo YAML de ejemplo permitiría ejecutar búsquedas completas sin escribir código Python, ampliando el acceso a la herramienta.

VI-E. Validación con Datasets Reales de Anomalías Industriales

Los experimentos de este trabajo usaron datasets sintéticos pensados para validar la correctitud funcional. Validar con datos reales de MVTec AD —con defectos industriales sutiles— permitiría medir el aporte real del preprocesamiento automático sobre modelos de referencia como IsolationForest u OC-SVM.

VI-F. Comparación con Herramientas AutoML Existentes

Comparar empíricamente el rendimiento y el tiempo de búsqueda de AutoPipeAD frente a herramientas AutoML generalistas como Auto-sklearn daría evidencia del valor diferencial de especializarse en detección de anomalías.

Referencias

[1] V. Chandola, A. Banerjee, y V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, sep. 2009.

[2] P. Bergmann, M. Fauser, D. Sattlegger, y C. Steger, "MVTec AD – A comprehensive real-world dataset for unsupervised anomaly detection," en *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, EE. UU., jun. 2019, pp. 9592–9600.

[3] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, y P. Gehler, "Towards total recall in industrial anomaly detection," en *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Nueva Orleans, LA, EE. UU., jun. 2022, pp. 14318–14328.

[4] T. Defard, A. Setkov, A. Loesch, y R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," en *Proc. Int. Conf. Pattern Recognit. (ICPR)*, Milán, Italia, ene. 2021, pp. 475–489.

[5] F. T. Liu, K. M. Ting, y Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 3:1–3:39, mar. 2012.

[6] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, y R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, jul. 2001.

[7] T. Akiba, S. Sano, T. Yanase, T. Ohta, y M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," en *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (KDD)*, Anchorage, AK, EE. UU., ago. 2019, pp. 2623–2631.

[8] J. Snoek, H. Larochelle, y R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," en *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, Lake Tahoe, NV, EE. UU., dic. 2012, pp. 2951–2959.

[9] L. Yang y A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, nov. 2020.

[10] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4.ª ed. Hoboken, NJ, EE. UU.: Pearson, 2018.

[11] T. Hastie, R. Tibshirani, y J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2.ª ed. Nueva York, NY, EE. UU.: Springer, 2009.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, y E. Duchesnay, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, oct. 2011.

[13] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, jun. 2006.

[14] A. Krizhevsky, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canadá, Inf. Téc., 2009.

[15] K. He, X. Zhang, S. Ren, y J. Sun, "Deep residual learning for image recognition," en *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, EE. UU., jun. 2016, pp. 770–778.